

What Native XDP Drop Benchmarks Omit: The Cost of a Minimal Packet-Reading Path

Luis Wehrberger
Hamburg University of Applied Sciences
Hamburg, Germany
luis.wehrberger@haw-hamburg.de

Abstract—Comparative packet-filter evaluations often represent native XDP by a minimal XDP_DROP program. Such a program drops packets without touching the packet itself. It is therefore not equivalent to a filter that classifies packets by their contents and must inspect at least one header field. The distinction matters because filtering options further down the receive path, such as generic XDP and Netfilter, run at a point where the relevant packet data may already have been brought into the processor’s cache. A native-XDP baseline that never touches the packet can thus omit a platform-dependent first-access cost that later hooks may not pay. Prior measurements of this cost differ by an order of magnitude and report no memory counters for the access itself. We measure the minimal read path on an AMD Zen 1 platform without a documented cache-injection mechanism, using an Intel 82599ES NIC with the ixgbe driver and a 512-descriptor RX ring, and compare the standard no-touch and read-data operations of xdp-bench at the native and generic hooks. Reading packet data lowers the native processing plateau from 14.29 to 10.12 Mpps and adds a median of 68 unhalted core cycles and 0.53 DRAM fills per processed packet. At the generic hook, the plateau changes only marginally, the median DRAM fill difference is zero, and the cycle difference is inconsistent across loads.

Index Terms—XDP, eBPF, packet filtering, packet-data access

I. INTRODUCTION

Performance evaluations commonly compare filtering mechanisms using minimal packet-drop benchmarks. Native XDP is frequently represented by a small program that immediately drops packets without inspecting them, while generic XDP or Netfilter discard the same traffic later in the Linux receive path [1], [2]. Although these programs ultimately take the same action, a program that does not access packet data is not a representative baseline, because a content-based filter must inspect at least one header field to match a rule.

The omitted access matters because its incremental cost can differ between hooks. Native XDP executes inside the NIC driver before the kernel allocates a socket buffer, so the program may be the first to touch a cache line written through DMA. Generic XDP and Netfilter run only after that point, by which time the relevant cache line may already have been brought closer to the core, either by the stack accessing the packet or by a driver prefetch that had time to complete. Header inspection can therefore impose a substantial cost on a first access in native XDP while adding little at later hooks.

How large this cost is depends on the platform. Two sources have measured it, and they differ substantially. Brouer

reports that reading packet data reduces native throughput by 39.4% [3], while the artifact of the original XDP evaluation reports a reduction of 5.3–7.0% [4]. Both attribute the cost to where the packet resides on the first access, and both report DDIO to be active. Brouer supports this with driver prefetching, which largely removes the penalty on his platform. Yet neither supports the attribution with counters for the two variants, and the platforms also differ in processor, NIC, driver, kernel, and configuration of the receive path. The explanation therefore remains untested.

We therefore address these research questions for a third platform without documented cache injection:

RQ1: How does the minimal packet-reading path affect processing capacity and per-packet cycle cost at the native and generic XDP hooks?

RQ2: From which source are the additional data-cache fills served that accompany the native packet read, and does the same read add fills at the generic hook?

We isolate the packet-data read at both hooks in a controlled measurement on a platform without documented cache injection and identify its fill source.

Based on prior measurements, we expect the minimal packet read to reduce capacity and increase per-packet cycles more at the native than the generic hook. Without documented cache injection, we further expect additional native DRAM-served data-cache fills and a smaller generic-hook difference.

Section II reviews the receive path and prior measurements. Section III describes the testbed and methodology, and Section IV reports the results. Section V interprets the results, relates them to prior platforms, and discusses implications and limitations. Section VI concludes.

II. BACKGROUND AND RELATED WORK

A. XDP and the eBPF Execution Environment

The eXpress Data Path (XDP) provides a programmable packet-processing layer within the Linux kernel. XDP programs run before packets enter the regular network stack and can selectively pass traffic to that stack when its functionality is required, providing an early processing path without replacing the host networking stack [1]. A program receives a context that provides access to the packet and its metadata, and returns an action that determines how the packet is handled.

XDP programs run in the kernel’s eBPF virtual machine, whose restricted instruction set and static verification make

it safe to execute user-supplied code in kernel context. The verifier requires an explicit bounds check before a header field can be read, and verified programs are then JIT-compiled to native machine instructions [1].

B. Filtering Hooks in the Linux Receive Path

Packets can be filtered at several points on their way from the network interface to the application, and the position of a filter determines how much processing a dropped packet has already caused [2]. Native XDP sits at the earliest of these software hooks. It runs inside the NIC driver on the DMA buffers, before the kernel allocates the socket buffer and other per-packet metadata, which is the main reason for its performance advantage [2]. That allocation separates the two hooks we compare: generic XDP acts on a packet the stack has already begun to process. By that point parts of the packet may already have been accessed or prefetched, which prior work suggests as a reason why the same access can be a first access at one hook and a repeated access at the other [3], [5]. Generic XDP is particularly suited for isolating this difference, since it executes the same eBPF program as native XDP at a later point in the receive path, without requiring driver support. It was introduced to ease development and testing rather than for throughput [3], so we treat it as a reference point for hook position rather than as a competing mechanism.

C. I/O Cache Placement

Where the processor finds incoming packet data on its first access depends on how the platform moves that data from the NIC [6]. With conventional Direct Memory Access (DMA), the operating system provides receive buffers and the NIC writes incoming packets to the corresponding addresses in main memory without involving the processor, which must subsequently fetch the required cache lines itself. Direct Cache Access (DCA) extends this path by prefetching selected parts of the I/O data into the processor cache, while the complete packet is still transferred to main memory. Intel’s Data Direct I/O (DDIO) changes the destination itself by allowing inbound DMA writes to allocate and update cache lines directly in the last-level cache. Whether such a line is still resident when the processor reads it is not guaranteed, since incoming packets compete for the limited portion of the cache used for I/O [6].

Independently of the transfer path, the driver may prefetch packet data before the XDP program runs, and whether this helps depends on timing. Paschali et al. observe that the program typically accesses the packet before the prefetch completes, and that delaying the access is enough for the line to arrive in time [5]. Where the packet resides at the XDP hook is therefore a property of the platform and of the receive path, not of the XDP program.

D. Prior Measurements of Packet-Data Access

Comparative evaluations of filtering mechanisms often follow two patterns. Some represent native XDP by a minimal program that drops packets without inspecting their contents. This establishes the throughput ceiling of the hook but omits

the access that any content-based filter must perform [1], [7]. Others evaluate filters that classify packets and therefore combine the first access with parsing and lookups [2], [8]. None of these cited comparisons isolates the cost of the first packet access by itself.

To the best of our knowledge, only two sources compare native-XDP drop programs directly with and without minimal packet-data access. Brouer reports that enabling a read of packet data reduces throughput from 19.80 to 12.01 Mpps, a reduction of 39.4% or approximately 32.8 ns per packet, whereas both variants reach approximately 8.4 Mpps at the generic hook [3]. He attributes the native penalty to the packet not yet being resident in a cache close to the core, supported by the observation that additional driver prefetching largely removes it. He also reports PMU observations indicating that DDIO was active on his platform, but DDIO support is not documented for the Intel Core i7-6700K used, so we treat the status of this platform as uncertain. The artifact accompanying the original XDP evaluation reports a much smaller reduction of 5.3% with 1024 RX descriptors and 7.0% with 512 descriptors, corresponding to approximately 2.3–3.0 ns per packet [4], measured on an Intel Xeon E5-1650 v4. The larger ring size thus yields the smaller reduction, although the artifact elsewhere reports misses at that size and attributes them to DDIO failing to keep the data in the last-level cache. The associated paper reports DDIO to be in use [1].

The two sources agree that packet-data access reduces native-XDP throughput but differ by an order of magnitude in its cost. Both report cache counters elsewhere in their measurements, yet neither reports them for the two variants: Brouer’s counters cover only the generic hook, and thus not the hook for which he offers the explanation, while the artifact compares cores within a multi-queue run rather than the two operations. The attribution to cache placement therefore rests on throughput alone.

Paschali et al. report the only cache measurement for this access that we found: approximately one L1 data-cache miss per packet for an XDP program that reads the packet header [5]. Their result does not determine whether that miss is served from another cache level or from main memory, nor does it compare the cost between the two hooks.

No measurement identifies the source of the accessed data or combines the same no-touch/read comparison at both hooks with paired cycle and fill counts.

III. EXPERIMENTAL METHODOLOGY

The experiment varies only packet access and repeats the paired comparison at native and generic XDP. Traffic, software, and CPU placement remain fixed. The artifact archives the scripts, exact versions, program disassemblies, validity runs, and raw records [9].

A. Compared Programs and Hooks

We compare the `no-touch` and `read-data` operations of the `xdp-bench` drop benchmark, run at both the native and the generic hook, which yields four configurations. Both

TABLE I
EXPERIMENTAL TESTBED.

Component	Configuration
DUT	AMD Ryzen 5 1600 (Zen 1), one 3.2GHz core
NIC	Intel X520 (82599ES), 10Gbit/s, <code>ixgbe</code>
Memory	2×8 GB DDR4-2133, one DIMM per channel
Caches	512 KB L2 per core, 8 MB L3 per CCX
Software	Ubuntu 26.04, Linux 7.0, <code>xdp-tools</code> 1.6.2
XDP path	JIT enabled, one RX queue with 512 descriptors
Generator	TRex 3.08 with DPDK, identical NIC
Traffic	64-byte frames, one UDP flow, constant rate

operations use the same benchmark infrastructure and per-packet statistics mechanism and return `XDP_DROP` for every packet. The `no-touch` operation reaches this verdict without accessing packet data. The `read-data` operation additionally validates the Ethernet-header bounds and reads the `EtherType` field before returning the same verdict. Within each hook, their paired difference isolates the complete added read path, and comparing the two differences shows how its cost changes with hook position. Using a standard benchmark keeps the comparison reproducible and avoids favouring either operation through implementation choices of our own.

We inspected the JIT output [9]. Reading the header adds 20 x86-64 instructions for the required bounds check, branch, and load. Both hooks use identical byte code, so differences reflect hook position rather than program code.

B. Testbed

Table I lists the components of the testbed and the settings that affect the measured receive path. Generator and device under test (DUT) are connected back-to-back over a dedicated 10 Gbit/s direct-attach copper link.

The DUT is configured so that the entire receive path for the offered traffic runs on one core, so that processing capacity stays below line rate. A single combined receive queue is used with the `ixgbe` default of 512 RX descriptors, and the queue interrupt is pinned to the measurement core. The core is isolated from scheduler and RCU work, its SMT sibling is offline, and the performance governor is active with boost and deep C-states disabled, so that cycle counts are not distorted by frequency changes or by unrelated work. Receive offloads and software steering are disabled, as is link-level flow control, so that overload manifests as packet loss at the DUT rather than as backpressure to the generator. We record missed and error counters for every trial.

We found no public documentation of a cache-injection mechanism for this platform. We nevertheless make no assumption about where a given cache line resides, since driver prefetching may still populate a processor cache before the XDP program runs.

C. Traffic and Run Procedure

TRex generates minimum-sized Ethernet frames from a single UDP flow, since these maximize the packet rate and therefore stress per-packet overhead rather than bandwidth. A

64-byte frame occupies 84 bytes on the wire once preamble, start delimiter, and inter-frame gap are counted, which puts the line rate of the 10 Gbit/s link at approximately 14.88 Mpps.

We sweep 0.25, 0.5, and 0.75 Mpps, followed by loads from 1 to 14.5 Mpps in steps of 0.5 Mpps, and add line rate as the final point, which yields 32 offered-load points. The three lowest rates give measurements below the saturation point of the generic hook, which the coarser steps would otherwise cover with a single point. These are the rates at which the generator is configured and by which we label results.

Each trial transmits traffic at a constant configured rate for 39 s, comprising a 5 s warm-up during which the receive path reaches steady state, a nominal 30 s measurement interval, and a 4 s post-measurement tail. We derive the realized offered rate from the generator’s transmit-packet counter over the complete 39 s traffic interval and repeat each configuration ten times. Comparable evaluations use measurement intervals of the same length [4], [2], [6].

We use medians to limit the influence of transient runs and report the interquartile range and observed minimum and maximum over ten repetitions. A paired difference is established when all ten differences share one sign and the two ranges do not overlap. An unestablished difference is not evidence of zero effect.

Each repetition contains one four-configuration block at every offered load. We randomize the offered-load order within each repetition and the configuration order within each block, so that slow drift is distributed across loads and configurations. We reapply the full configuration before each block and record and verify DUT state around every window.

We validate the offered load in two ways. TRex derives the transmitted rate from its TX counter and configured traffic duration, and we independently derive the arrival rate from `rx_packets` plus `rx_missed_errors` over the time-stamped DUT counter window. The two agree to a median deviation of 0.001%, and to 0.04% at the 99th percentile, across all valid trials. A separate validation confirms that the generator sustains the requested load up to line rate without transmit errors [9].

D. Metrics

The configured generator rate labels each offered-load point. We derive the realized offered load from the generator’s transmit-packet count over the 39 s traffic interval. The driver counts every packet removed from the RX ring, including packets subsequently dropped by XDP, in `rx_packets`. Packets discarded by the NIC before reaching the ring are reported as `rx_missed_errors`.

Our first metric is the number of unhalted core cycles the receive path spends on one processed packet. Two counters produce it, and they do not cover exactly the same interval. The NIC counters bracket the performance counters, so their window runs 48 to 68 ms longer than the roughly 30 s that the performance counters cover.

A rate bridges that gap. Dividing `rx_packets` by the NIC-counter window gives the processed rate λ , and multi-

plying it by the performance-counter window T_{perf} gives the packets processed while the performance counters ran:

$$C = \frac{N_{\text{cycles}}}{\lambda T_{\text{perf}}},$$

where N_{cycles} is the count on the measurement core. The longer NIC window holds at most 0.22% more packets than the performance-counter window, so using `rx_packets` directly would lower C by that much. We count unhalted core cycles with AMD’s documented core-cycle event. We cross-check each window against `APERF`, the processor’s independent counter for actual performance-frequency cycles [10].

This metric deliberately covers the complete receive path rather than the XDP program alone. It includes interrupt handling, driver and NAPI work, and the XDP program. In generic mode it additionally includes socket-buffer allocation and preceding stack processing. Absolute values are therefore not comparable across hooks.

Per-program profiling can reduce packet rate and delay the access under study. Prior work shows that such delays can change the observed cache state [5]. We therefore program the counters once per measurement window and read them after it closes. This avoids per-packet instrumentation. Per-program BPF runtime accounting remains disabled for the same reason.

Our second metric, F , is the number of demand data-cache fills per processed packet, classified by source. We use AMD’s documented demand data-cache fill event with the three source categories applicable to our single-die DUT: the local L2, the local cache complex, and local DRAM [10]. A pointer-chase microbenchmark verified these assignments on the DUT [9]. We divide each count by the packets processed in the same window. All events ran together on the isolated core and counted user and kernel execution. Each event covered the full measurement window, so no counter was multiplexed or scaled. The artifact records the exact encodings [9].

For any per-packet quantity X , we write

$$\Delta X = X_{\text{read-data}} - X_{\text{no-touch}}$$

for its paired difference at one hook, offered load, and repetition. We estimate the packet-read cost from ΔC , and the memory traffic that accompanies it from ΔF for each source.

For each hook, offered load, and repetition, the `read-data` observation is paired with the `no-touch` observation from the same four-configuration block.

Both programs also update the benchmark’s BPF-map counter. We leave this update enabled because disabling it would require modifying the benchmark program. The update is part of the measured path, but the counter is not used in our metric. Instead, we use its reported rate to validate `rx_packets`. Across valid trials, the rates agree within 0.002% at the median and 0.10% at the 99th percentile.

E. Saturation Criterion

Prior packet-processing evaluations use loss thresholds of 0.1% and 1%. We adopt the stricter [11], [8]. For each configuration and offered-load point, we compute the median

of the loss fractions from all valid repetitions. The load point is classified as processing-limited when

$$\text{median}(L_1, \dots, L_n) > 0.1\%.$$

For each configuration, we define the processing plateau as the mean processed load across all blocks at processing-limited loads. The plateau is the one quantity for which we take the mean rather than the median. It has to state a capacity, so it must match the work the configuration actually did. All windows have the same length, so the mean equals the packets processed divided by the time taken, whereas a median would not. The number of such loads differs between configurations.

F. Use of Generative AI

We used Anthropic’s Claude Opus 5 in two roles. First, it assisted in writing the scripts of this study, including those that run the measurement campaign, aggregate the per-cell records, and produce Figures 1 and 2. The author specified each script, revised its output, and validated reported values against the raw records. All scripts are archived with the artifact [9]. Second, it served as a critic during the design of the study, raising objections to metric definitions and validity arguments. The author decided the experimental design, the metric definitions, and the testbed configuration, and verified every point raised in this way against the primary sources and the raw records.

IV. RESULTS

The campaign covers 32 offered loads, 10 blocks per load, and 4 configurations per block, which yields 1280 cells. The raw records, the analysis scripts, and the exact versions are archived with the artifact [9].

A. Processing Plateaus

We first classify every offered load by whether its configuration is processing-limited, and then read the plateau from the classified loads. Figure 1 plots processed load against offered load. Native `no-touch` follows the offered load without loss through 14.0 Mpps and becomes processing-limited at 14.5 Mpps. Native `read-data` becomes processing-limited at 10.0 Mpps, and both generic configurations at 1.5 Mpps.

The native `no-touch` plateau covers 2 loads and 20 blocks and lies at 14.290 Mpps. The native `read-data` plateau covers 11 loads and 110 blocks and lies at 10.123 Mpps. Each plateau is a mean over its classified blocks. The two differ by 4.17 Mpps, which is 29.2% of the `no-touch` plateau. At the two loads where both native configurations are processing-limited, `read-data` processes less in all 20 pairs, and the two ranges stay disjoint, so the native capacity difference is established. Its per-load medians vary by up to 1.42 Mpps between neighbouring loads without a trend. At 14.0 Mpps, the ten `read-data` blocks range from 8.08 to 12.50 Mpps, a spread of 4.42 Mpps.

Each generic plateau covers 28 loads and 280 blocks. The two lie at 1.246 and 1.244 Mpps and differ by 0.0023 Mpps, or 0.18%. Their ranges overlap at all 28 shared loads, so the generic capacity difference is not established.

TABLE II
PER-PACKET LEVELS AT THE TWO LOADS WHERE ALL FOUR
CONFIGURATIONS ARE PROCESSING-LIMITED, 14.5 MPPS AND LINE RATE.
EACH VALUE IS A MEDIAN OVER 20 BLOCKS.

Hook	Operation	Cycles	Fills		
			L2	CCX	DRAM
native	no-touch	223.7	0.5500	0.0620	0.0000
native	read-data	294.0	0.4141	0.0799	0.3640
generic	no-touch	2563.0	6.1722	0.4224	0.0639
generic	read-data	2563.4	6.1299	0.3783	0.0640

B. Per-Packet Cycle Cost

We next measure what the read costs per packet at each hook, and whether the paired difference counts as established. Every difference below is a median over all 32 loads and 10 blocks, which is 320 pairs per hook. The first panel of Figure 2 reports the cycle difference at each offered load. At the native hook the difference is 67.9 cycles per packet. It is positive in all 320 pairs and established at all 32 loads.

At the generic hook the difference is 4.6 cycles per packet. It is positive in 249 of 320 pairs and established at none of the 32 loads. Its per-load medians run from -3.3 to $+8.1$ cycles per packet, so the difference changes sign across the sweep.

The native difference is 14.8 times the generic one and exceeds it by 63.4 cycles per packet. Table II puts both against their own baseline at the two loads where all four configurations are processing-limited. Native *no-touch* runs at 223.7 cycles per packet and generic *no-touch* at 2563.0, so the generic difference amounts to 0.18% of its own baseline.

C. Source of the Additional Fills

We finally ask which source serves the additional fills at the native hook, and whether the same read adds fills at the generic hook. The last three panels of Figure 2 report the fill difference by source. At the native hook local DRAM adds a median 0.529 fills per packet over the 320 pairs, positive in all of them and established at all 32 loads. That difference depends on the offered load. Its per-load median peaks at 0.804 fills per packet at 7.0 Mpps and falls to 0.260 at line rate, so the two loads in Table II sit below it at 0.364. The local cache complex adds 0.011 fills per packet and is established at 1 load. The local L2 adds 0.004 fills per packet and is established at no load. Local DRAM is therefore the only established source.

At the generic hook no source is established at any load. The DRAM difference has a median of zero and ranges from -0.0023 to $+0.0037$ fills per packet.

The levels in Table II separate the two hooks more sharply than the differences do. Native *no-touch* records 0.0000 DRAM fills per packet in all 20 blocks, while native *read-data* records 0.364. Both generic configurations record about 0.064 whether or not the program reads.

V. DISCUSSION

A. The Cost of the First Access

RQ1 asks what the minimal read path costs at each hook. At the native hook it costs 29.2% of the processing capacity

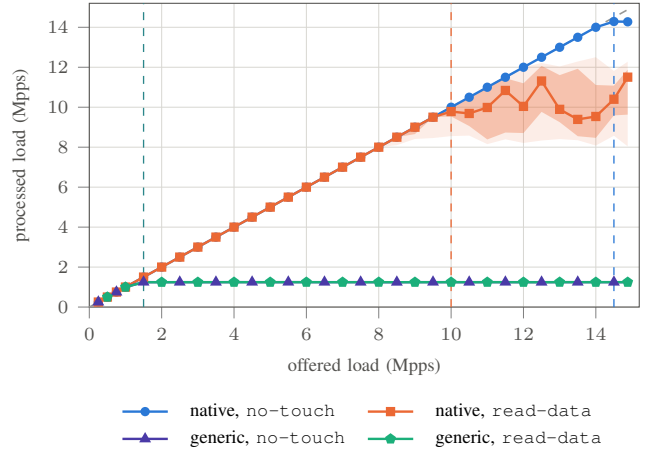


Fig. 1. Processed load for the four configurations. Points show the median of ten repetitions. Dark bands show the interquartile range, and light bands show the observed min-max range. Vertical dashed lines mark where each configuration becomes processing-limited.

and an established 67.9 cycles per packet, which is 21.3 ns at the measured core clock. At the generic hook neither the capacity nor cycle difference is established under this descriptive criterion: the median cycle difference is 4.6 cycles per packet, but its per-load medians change sign across the sweep. The observed median difference there stays within a fraction of a percent of the 2563 cycles the generic path already spends.

RQ2 asks which source serves the additional data-cache fills that accompany the native packet read and whether the same read adds fills at the generic hook. At the native hook, local DRAM is the only established source. At the generic hook, the same read adds no fills that meet the established criterion. At the plateau, native *no-touch* records no DRAM fills, whereas *read-data* records 0.364 per packet. Overall, this hook contrast is consistent with the qualitative expectation in Section I.

Neither native value reaches one DRAM fill per packet, so not every packet read causes a fill. We count demand fills, so a line that a prefetch brings in ahead of the program's load does not appear. The value also varies across offered loads, and our study does not isolate why.

The generic number is small in absolute terms as well. About one packet in sixteen records a DRAM fill on that path. The most plausible reason is timing: the path spends longer on a packet, which leaves a prefetch more room to finish. Paschali et al. observed a similar effect [5].

B. Relation to Prior Measurements

Our 29.2% native reduction and 21.3 ns cost fall between Brouer's 39.4% and 32.8 ns and the original artifact's 7.0% and 2.3 to 3.0 ns [3], [4]. These measurements show a wide cross-platform spread. At the generic hook, our result agrees with Brouer's finding of similar throughput for both variants [3].

Paschali et al. report approximately one L1 data-cache miss per packet for a header-reading XDP program [5]. Native

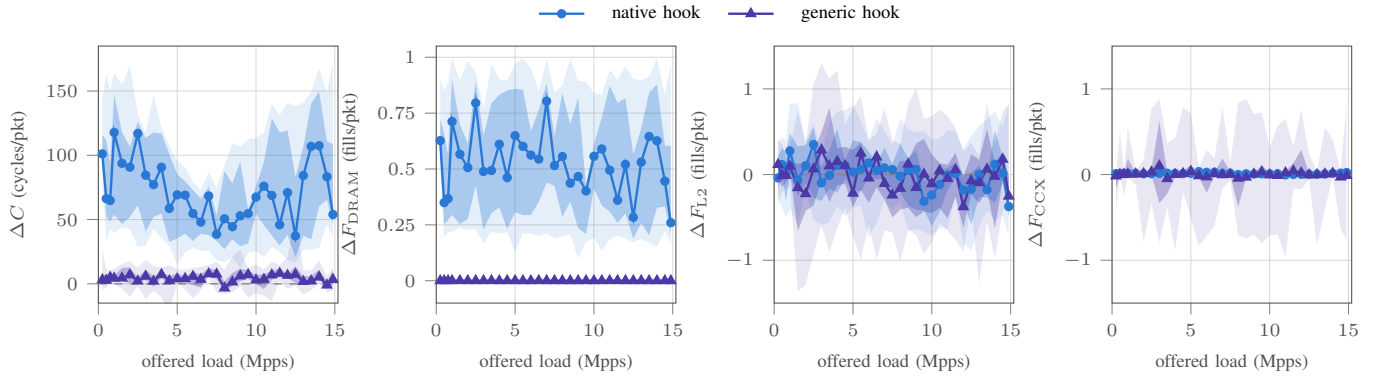


Fig. 2. Within-block packet-read differences (`read-data` minus `no-touch`). Points show ten-repetition medians. Dark and light bands show interquartile and observed ranges. Panels report cycles and fills served by local DRAM, L2, and cache complex.

`read-data` records 0.85 fills per packet in total here. Their measurement does not resolve which level served the miss. We resolve it for one platform without a documented cache-injection mechanism, where the answer is main memory.

C. Implications for Packet-Filter Benchmarks

On this platform the read costs 29.2% of the native capacity and does not establish a comparable cost at the generic hook. That only distorts a comparison when the native side does not read. A benchmark whose native program filters on packet contents already pays the access, and nothing is missing. A benchmark that compares native `no-touch` with a reading filter at a later hook credits native XDP twice: for the earlier hook and for an omitted first access.

Evaluations should report a minimally reading baseline next to the `no-touch` ceiling and document the processor, NIC, driver, kernel, RX-ring size, prefetch behaviour, and cache-injection mechanism. All can affect the measured reduction.

D. Limitations

The results describe one platform. The size of the effect follows that platform. The NIC, driver, RX-ring size, prefetch behaviour, and cache-injection mechanism may affect it. Native `no-touch` is processing-limited only at the final two loads, so its plateau and the shared-load capacity comparison rely on 20 blocks at two load points. This limits the assessment despite a consistent gap in all 20 pairs.

The cause of the native processing-limited native `read-data` scatter remains unexplained, as does the variation in its DRAM fills per packet.

The counters cover the complete receive path. Paired differences remove the work that the two operations share, but they do not isolate the program itself. The measurement core also shares its 8 MB L3 cache with two other online cores, and activity on those cores may have affected the fill counts.

VI. CONCLUSION

Minimal native-XDP drop benchmarks omit the first packet-data access required by content-based filters. On our AMD Zen 1 platform, reading one header field lowers the native

processing plateau by 29.2% and adds 68 unhalting core cycles per packet. Local DRAM is the only established source of the additional fills, with a median of 0.53 fills per packet. At the generic hook, the corresponding capacity and cycle changes are small, and the median fill difference is zero. Comparing native `no-touch` with a later filter can therefore credit native XDP for the earlier hook and an omitted access.

REFERENCES

- [1] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, and D. Miller, “The eXpress data path: Fast programmable packet processing in the operating system kernel,” in *Proceedings of the 14th International Conference on Emerging Networking Experiments and Technologies*, ser. CoNEXT ’18. New York, NY, USA: Association for Computing Machinery, Dec. 2018, pp. 54–66. [Online]. Available: <https://doi.org/10.1145/3281411.3281443>
- [2] D. Scholz, D. Raumer, P. Emmerich, A. Kurtz, K. Lesiak, and G. Carle, “Performance implications of packet filtering with linux ebpf,” in *2018 30th International Teletraffic Congress (ITC 30)*, vol. 01, 2018, pp. 209–217.
- [3] J. D. Brouer, “Eval generic netstack XDP patch,” Prototype Kernel Documentation, Apr. 2017, version 1.0.1, updated June 8, 2017; accessed August 21, 2026. [Online]. Available: https://prototype-kernel.readthedocs.io/en/latest/blogposts/xdp25_eval_generic_xdp_tx.html
- [4] T. Høiland-Jørgensen and J. D. Brouer, “XDP benchmark baseline,” Benchmark artifact accompanying the CoNEXT 2018 XDP paper, 2018, file: `bench01_baseline.org`; accessed August 21, 2026. [Online]. Available: https://github.com/tohojo/xdp-paper/blob/master/benchmarks/bench01_baseline.org
- [5] V. Paschali, A. Monterubbiano, F. Fazzari, M. Swift, and S. Pontarelli, “InXpect: Lightweight XDP profiling,” in *Proceedings of the 3rd Workshop on eBPF and Kernel Extensions*, ser. eBPF ’25. New York, NY, USA: Association for Computing Machinery, Sep. 2025. [Online]. Available: <https://doi.org/10.1145/3748355.3748367>
- [6] A. Farshin, A. Roozbeh, G. Q. M. Jr., and D. Kostić, “Reexamining direct cache access to optimize I/O-intensive applications for multi-hundred-gigabit networks,” in *2020 USENIX Annual Technical Conference*, ser. USENIX ATC ’20. Boston, MA, USA: USENIX Association, Jul. 2020, pp. 673–689. [Online]. Available: <https://www.usenix.org/conference/atc20/presentation/farshin>
- [7] O. Hohlfeld, J. Krude, J. H. Reelfs, J. Rütth, and K. Wehrle, “Demystifying the performance of XDP BPF,” in *2019 IEEE Conference on Network Softwarization (NetSoft)*. IEEE, 2019, pp. 208–212. [Online]. Available: <https://doi.org/10.1109/NETSOFT.2019.8806651>
- [8] S. Miano, M. Bertrone, F. Risso, M. Vázquez Bernal, Y. Lu, and J. Pi, “Securing Linux with a faster and scalable iptables,” *ACM SIGCOMM Computer Communication Review*, vol. 49, no. 3, pp. 2–17, 2019.
- [9] L. Wehrberger, “Artifact for “What Native XDP Drop Benchmarks Omit: The Cost of a Minimal Packet-Reading Path,”,” GitHub repository, 2026. [Online]. Available: <https://github.com/luisluis8414/What-Native-XDP-Drop-Benchmark-s-Omit-The-Cost-of-a-Minimal-Packet-Reading-Path>
- [10] *Processor Programming Reference (PPR) for AMD Family 17h Models 01h,08h, Revision B2 Processors*, Advanced Micro Devices, Jun. 2019, publication 54945, Revision 3.03, Section 2.1.15.4.2 (LS Events).
- [11] F. Parola, R. Procopio, R. Querio, and F. Risso, “Comparing user space and in-kernel packet processing for edge data centers,” *ACM SIGCOMM Computer Communication Review*, vol. 53, no. 1, pp. 14–29, 2023.